

Ejercicio 7: Mezcla Óptima de Ficheros mediante Algoritmos Voraces

Este documento presenta la resolución matemática y el diseño del algoritmo **Voraz** (*Greedy*) óptimo para mezclar un conjunto de n ficheros ordenados de longitudes conocidas de manera que se minimice el número total de comparaciones de elementos.

1. Caracterización del Problema

Cuando mezclamos dos ficheros ordenados de longitudes L_1 y L_2 para generar un tercer fichero ordenado de longitud $L_1 + L_2$, el coste del algoritmo de mezcla (en número de comparaciones en el peor caso) es lineal respecto a la suma de sus longitudes:

$$\text{Coste}(L_1, L_2) = \Theta(L_1 + L_2)$$

Al tener n ficheros iniciales, debemos realizar $n - 1$ mezclas sucesivas de dos en dos hasta obtener un único fichero final de tamaño $\sum L_i$. Cada forma de emparejar los ficheros se puede modelar como un **árbol binario de mezcla**, donde:

- Las hojas representan los n ficheros originales.
- Los nodos internos representan las mezclas intermedias.
- El coste de mezclar un fichero original i de longitud L_i es proporcional a la cantidad de veces que se procesa, es decir, su profundidad d_i en el árbol de decisión.

Por tanto, queremos construir un árbol que minimice el coste de mezcla total (ponderado por profundidad):

$$\text{Minimizar Coste Total} = \sum_{i=1}^n L_i \cdot d_i$$

2. Estrategia Algorítmica (Elección Voraz)

Para minimizar el sumatorio anterior, los ficheros con mayor longitud L_i deben estar lo más cerca posible de la raíz del árbol (menor profundidad d_i), mientras que los ficheros más pequeños pueden situarse a mayor profundidad.

Propiedad de la Elección Voraz

La estrategia voraz consiste en:

1. Identificar, de entre todos los ficheros (tanto originales como subficheros ya resultantes de mezclas previas), los dos ficheros con las longitudes más pequeñas.
2. Mezclarlos para crear un nuevo subfichero cuya longitud será la suma de ambos.

3. Repetir el proceso con el conjunto de ficheros restante hasta que solo quede un único fichero unificado.

3. Algoritmo en Pseudocódigo

```

Funcion CalcularMezclaOptimaFicheros(L, n)
Inicio
    // L es un vector que contiene las longitudes de los n ficheros iniciales
    Q <- CrearMinHeap() // Cola con prioridad de mínimo

    // 1. Insertar todos los tamaños de los ficheros en la cola de prioridad
    Para i <- 1 Hasta n Con Paso 1 Hacer
        Insertar(Q, L[i]) // Coste de construcción del Heap: O(n) o O(n log n)
    FinPara

    coste_total_comparaciones <- 0

    // 2. Realizar n-1 mezclas sucesivas de los dos ficheros más pequeños
    Mientras Tamaño(Q) > 1 Hacer
        // Extraer los dos ficheros con las longitudes más pequeñas
        f_primero <- ExtraerMinimo(Q) // Coste: O(log n)
        f_segundo <- ExtraerMinimo(Q) // Coste: O(log n)

        // El coste de mezclar estos dos ficheros es la suma de sus longitudes
        coste_mezcla_actual <- f_primero + f_segundo
        coste_total_comparaciones <- coste_total_comparaciones + coste_mezcla_act

        // Reinsertar el nuevo subfichero resultante en la cola de prioridad
        Insertar(Q, coste_mezcla_actual) // Coste: O(log n)
    FinMientras

    Imprimir("Número mínimo de comparaciones requeridas: ", coste_total_comparaciones)
    Retornar coste_total_comparaciones
FinFuncion

```

4. Análisis de Coste

- **Complejidad Temporal:** $O(n \log n)$.
 - Insertar los n elementos iniciales en la cola con prioridades requiere un tiempo de $O(n \log n)$ (o $O(n)$ si se utiliza el algoritmo de linealización *Heapify*).
 - El bucle `Mientras` se ejecuta exactamente $n - 1$ veces. En cada iteración se realizan dos operaciones de extracción del mínimo y una inserción. Cada una de estas operaciones tiene un coste logarítmico de $O(\log n)$ en el min-heap.
 - Por tanto, el coste de procesamiento del bucle domina la complejidad temporal total: $T(n) \in O(n \log n)$.
- **Complejidad Espacial:** $O(n)$ para almacenar las longitudes de los ficheros activos e intermedios en la estructura del min-heap.

5. Justificación de Corrección (Demostración de Optimalidad)

Para demostrar que el algoritmo voraz es correcto y encuentra el árbol de coste mínimo, utilizaremos un **argumento de intercambio** (*Exchange Argument*).

Lema del Paso Voraz

Sean f_1 y f_2 las longitudes de los dos ficheros más pequeños del conjunto. Existe un árbol de mezcla óptimo donde f_1 y f_2 son nodos hermanos situados en la máxima profundidad del árbol.

Demostración del Lema:

1. Supongamos que existe un árbol de mezcla óptimo T donde los dos ficheros más pequeños, f_1 y f_2 , no son hermanos o no están en el nivel de máxima profundidad.
2. Sea x e y una pareja de hojas que son hermanas en el nivel de máxima profundidad de T ($d_x = d_y = d_{\max}$).
3. Sin pérdida de generalidad, asumimos que $f_1 \leq f_2 \leq x \leq y$ (pues f_1 y f_2 son los dos elementos mínimos de todo el conjunto).
4. Si intercambiamos la posición de f_1 con la de x para obtener un nuevo árbol T' , la variación en el coste de mezcla total ΔCoste será:

$$\Delta\text{Coste} = (f_1 \cdot d_x + x \cdot d_{f_1}) - (f_1 \cdot d_{f_1} + x \cdot d_x)$$

$$\Delta\text{Coste} = (x - f_1)(d_{f_1} - d_x)$$

5. Como $d_x = d_{\max}$ es la profundidad máxima de todo el árbol, se cumple que $d_{f_1} \leq d_x \implies (d_{f_1} - d_x) \leq 0$. Además, por hipótesis, $(x - f_1) \geq 0$. Por tanto:

$$\Delta\text{Coste} \leq 0$$

6. Esto demuestra que realizar el intercambio no puede incrementar el coste del árbol. Al contrario, o lo mantiene igual (si tenían los mismos valores o profundidades) o lo reduce.
7. Si realizamos el mismo intercambio para situar a f_2 en la posición del nodo y , el coste del nuevo árbol T'' volverá a cumplir que $\Delta\text{Coste} \leq 0$.
8. Como el árbol original T ya era óptimo por hipótesis (su coste era el mínimo absoluto), el coste no puede disminuir por debajo del óptimo. Por tanto, el coste de T'' debe ser exactamente igual al de T .

Esto demuestra que la decisión del algoritmo voraz de emparejar primero los dos elementos más pequeños en la máxima profundidad siempre es óptima y forma parte de un árbol de coste mínimo global. Por inducción matemática sobre el número de ficheros n , la optimalidad se mantiene en cada una de las mezclas sucesivas del algoritmo.